



langgraph-supervisor

v0.0.31

license MIT

downloads 3.1M



Note

We now recommend using the **supervisor pattern directly via tools** rather than this library for most use cases. The tool-calling approach gives you more control over context engineering and is the recommended pattern in the [LangChain multi-agent guide](#).

See our [supervisor tutorial](#) for a step-by-step guide.

We're making this library compatible with LangChain 1.0 to help users upgrade their existing code. If you find this library solves a problem that can't be easily addressed with the manual supervisor pattern, we'd love to hear about your use case!

See the [project description](#) for more details.

mod supervisor

FUNCTION

DESCRIPTION

`create_supervisor`

Create a multi-agent supervisor.

func create_supervisor

```
create_supervisor(  
    agents: List[Pregel],  
    *,  
    model: LanguageModelLike,  
    tools: List[BaseTool | Callable] | ToolNode | None = None,  
    prompt: Prompt | None = None,  
    response_format: StructuredResponseSchema  
    | tuple[str, StructuredResponseSchema]  
    | None = None,  
    pre_model_hook: RunnableLike | None = None,  
    post_model_hook: RunnableLike | None = None,  
    parallel_tool_calls: bool = False,  
    state_schema: StateSchemaType | None = None,  
    context_schema: Type[Any] | None = None,  
    output_mode: OutputMode = "last_message",  
)
```

```

add_handoff_messages: bool = True,
handoff_tool_prefix: str | None = None,
add_handoff_back_messages: bool | None = None,
supervisor_name: str = "supervisor",
include_agent_name: AgentNameMode | None = None,
**deprecated_kwargs: Unpack[DeprecatedKwargs],
) -> StateGraph

```

Create a multi-agent supervisor.

PARAMETER	DESCRIPTION
<code>agents</code>	List of agents to manage. An agent can be a LangGraph <code>CompiledStateGraph</code>, a functional API workd any other <code>Pregel</code> object. TYPE: <code>List[Pregel]</code>
<code>model</code>	Language model to use for the supervisor TYPE: <code>LanguageModelLike</code>
<code>tools</code>	Tools to use for the supervisor TYPE: <code>List[BaseTool Callable] ToolNode None</code> DEFAULT
<code>prompt</code>	Optional prompt to use for the supervisor. Can be one of: <code>str</code>: This is converted to a <code>SystemMessage</code> and added to the beginning list of messages in <code>state["messages"]</code>. <code>SystemMessage</code>: this is added to the beginning of the list of messages <code>state["messages"]</code>. <code>Callable</code>: This function should take in full graph state and the output passed to the language model. <code>Runnable</code>: This runnable should take in full graph state and the output passed to the language model. TYPE: <code>Prompt None</code> DEFAULT
<code>response_format</code>	An optional schema for the final supervisor output. If provided, output will be formatted to match the given schema and return the <code>'structured_response'</code> state key. If not provided, <code>structured_response</code> will not be present in the output stat

PARAMETER

DESCRIPTION

Can be passed in as:

- An OpenAI function/tool schema,
- A JSON Schema,
- A TypedDict class,
- A Pydantic class.
- A tuple `(prompt, schema)`, where schema is one of the above. The prompt can be used together with the model that is being used to generate the structured response.

 Important

`response_format` requires the model to support `.with_structured_output`

 Note

`response_format` requires `structured_response` key in your state schema

You can use the prebuilt

`langgraph.prebuilt.chat_agent_executor.AgentStateWithStructuredResponse`

TYPE: `StructuredResponseSchema` | `tuple[str, StructuredResponseSchema]`
DEFAULT: `None`

`pre_model_hook`

An optional node to add before the LLM node in the supervisor agent (i.e., the node that calls the LLM).

Useful for managing long message histories (e.g., message trimming, summarization, etc.).

Pre-model hook must be a callable or a runnable that takes in current graph state and returns a state update in the form of

```
# At least one of `messages` or `llm_input_messages` MUST be provided
{
    # If provided, will UPDATE the `messages` in the state
    "messages": [RemoveMessage(id=REMOVE_ALL_MESSAGES), ...],
    # If provided, will be used as the input to the LLM,
    # and will NOT UPDATE `messages` in the state
    "llm_input_messages": [...],
    # Any other state keys that need to be propagated
    ...
}
```

PARAMETER	DESCRIPTION
	 Important At least one of <code>messages</code> or <code>llm_input_messages</code> MUST be provided and be used as an input to the <code>agent</code> node. The rest of the keys will be added to the graph state.  Warning If you are returning <code>messages</code> in the pre-model hook, you should OVERWRITE the <code>messages</code> key by doing the following: <pre>{ "messages": [RemoveMessage(id=REMOVE_ALL_MESSAGES), *new_messages] ...}</pre> TYPE: <code>RunnableLike</code> <code>None</code> DEFAULT: <code>None</code>
<code>post_model_hook</code>	An optional node to add after the LLM node in the supervisor agent (i.e., the node that calls the LLM). Useful for implementing human-in-the-loop, guardrails, validation, or other processing. Post-model hook must be a callable or a runnable that takes in current graph state and returns a state update. TYPE: <code>RunnableLike</code> <code>None</code> DEFAULT: <code>None</code>
<code>parallel_tool_calls</code>	Whether to allow the supervisor LLM to call tools in parallel (only OpenAI and Anthropic). Use this to control whether the supervisor can hand off to multiple agents. If <code>True</code>, will enable parallel tool calls. If <code>False</code>, will disable parallel tool calls.

PARAMETER	DESCRIPTION
	 Important This is currently supported only by OpenAI and Anthropic models. To call parallel tool calling for other providers, add explicit instructions for tool use in the system prompt. TYPE: <code>bool</code>DEFAULT:
<code>state_schema</code>	State schema to use for the supervisor graph. TYPE: <code>StateSchemaType</code> <code>None</code>DEFAULT:
<code>context_schema</code>	Specifies the schema for the context object that will be passed to the world model. TYPE: <code>Type[<i>Any</i>]</code> <code>None</code>DEFAULT:
<code>output_mode</code>	Mode for adding managed agents' outputs to the message history in the next agent workflow. Can be one of: <code>full_history</code>: Add the entire agent message history <code>last_message</code>: Add only the last message TYPE: <code>OutputMode</code>DEFAULT: <code>'last_message'</code>
<code>add_handoff_messages</code>	Whether to add a pair of (<code>AIMessage</code>, <code>ToolMessage</code>) to the message history when a handoff occurs. TYPE: <code>bool</code>DEFAULT:
<code>handoff_tool_prefix</code>	Optional prefix for the handoff tools (e.g., <code>'delegate_to_'</code> or <code>'transfer_to_'</code>). If provided, the handoff tools will be named <code>handoff_tool_prefix_agent_name</code>. If not provided, the handoff tools will be named <code>transfer_to_agent_name</code>. TYPE: <code>str</code> <code>None</code>DEFAULT:
<code>add_handoff_back_messages</code>	Whether to add a pair of (<code>AIMessage</code>, <code>ToolMessage</code>) to the message history when returning control to the supervisor to indicate that a handoff has occurred. TYPE: <code>bool</code> <code>None</code>DEFAULT:

PARAMETER	DESCRIPTION
<code>supervisor_name</code>	Name of the supervisor node. TYPE: <code>str</code> DEFAULT: <code>'supervisor'</code>
<code>include_agent_name</code>	Use to specify how to expose the agent name to the underlying supervisor. <code>None</code> : Relies on the LLM provider using the name attribute on the AI message. Currently, only OpenAI supports this. <code>'inline'</code> : Add the agent name directly into the content field of the AI message using XML-style tags. Example: <code>"How can I help you"</code> → <code>"<name>agent_name</name><content>How can I help you?</content>"</code> TYPE: <code>AgentNameMode</code> <code>None</code> DEFAULT: <code>None</code>

**Example**

```
from langchain_openai import ChatOpenAI

from langgraph_supervisor import create_supervisor
from langgraph.prebuilt import create_react_agent

# Create specialized agents

def add(a: float, b: float) -> float:
    '''Add two numbers.'''
    return a + b

def web_search(query: str) -> str:
    '''Search the web for information.'''
    return 'Here are the headcounts for each of the FAANG companies in 2024...'

math_agent = create_react_agent(
    model="openai:gpt-4o",
    tools=[add],
    name="math_expert",
)

research_agent = create_react_agent(
    model="openai:gpt-4o",
    tools=[web_search],
    name="research_expert",
)

# Create supervisor workflow
workflow = create_supervisor(
    [research_agent, math_agent],
    model=ChatOpenAI(model="gpt-4o"),
)

# Compile and run
app = workflow.compile()
result = app.invoke({
    "messages": [
        {
            "role": "user",
            "content": "what's the combined headcount of the FAANG companies in 2024?"
        }
    ]
})
```

mod handoff

FUNCTION	DESCRIPTION
<code>create_handoff_tool</code>	Create a tool that can handoff control to the requested agent.

FUNCTION	DESCRIPTION
<code>create_forward_message_tool</code>	Create a tool the supervisor can use to forward a worker message by name.

`func` `create_handoff_tool`

```

create_handoff_tool(
    *,
    agent_name: str,
    name: str | None = None,
    description: str | None = None,
    add_handoff_messages: bool = True,
) -> BaseTool

```

Create a tool that can handoff control to the requested agent.

PARAMETER	DESCRIPTION
<code>agent_name</code>	The name of the agent to handoff control to, i.e. the name of the agent node in the multi-agent graph. Agent names should be simple, clear and unique, preferably in snake_case, although you are only limited to the names accepted by LangGraph nodes as well as the tool names accepted by LLM providers (the tool name will look like this: <code>transfer_to_<agent_name></code>). TYPE: <code>str</code>
<code>name</code>	Optional name of the tool to use for the handoff. If not provided, the tool name will be <code>transfer_to_<agent_name></code>. TYPE: <code>str</code> <code>None</code> DEFAULT: <code>None</code>
<code>description</code>	Optional description for the handoff tool. If not provided, the description will be <code>Ask agent <agent_name> for help</code>. TYPE: <code>str</code> <code>None</code> DEFAULT: <code>None</code>

PARAMETER	DESCRIPTION
<code>add_handoff_messages</code>	Whether to add handoff messages to the message history. If <code>False</code> , the handoff messages will be omitted from the message history. TYPE: <code>bool</code> DEFAULT: <code>True</code>

`func` `create_forward_message_tool`

```
create_forward_message_tool(supervisor_name: str = 'supervisor') -> BaseTool
```



Create a tool the supervisor can use to forward a worker message by name.

This helps avoid information loss any time the supervisor rewrites a worker query to the user and also can save some tokens.

PARAMETER	DESCRIPTION
<code>supervisor_name</code>	The name of the supervisor node (used for namespacing the tool). TYPE: <code>str</code> DEFAULT: <code>'supervisor'</code>

RETURNS	DESCRIPTION
<code>BaseTool</code>	The <code>'forward_message'</code> tool. TYPE: <code>BaseTool</code>